

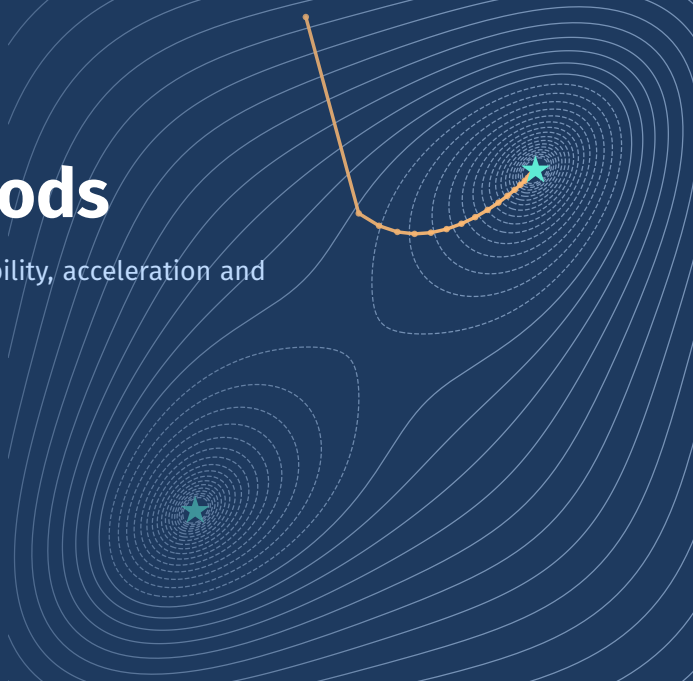
CSE 402 • NUMERICAL ANALYSIS • TOPIC 3

Gradient Methods

From “slope $\times$ learning rate” to stability, acceleration and SGD

Nawriz Ahmed Turjo

Roll 2105032 • October 2026



Topic 3: Gradient Methods

The same update fits a line by hand and trains large models.

How it works (1–4)

Fitting a line, the gradient, the update rule, and watching GD move on contours and in 3D.

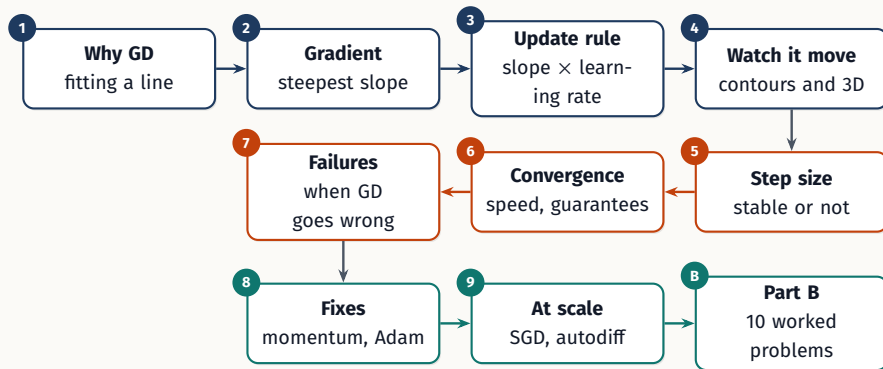
When it works, and when not (5–9)

The learning rate and stability, speed and guarantees, failure modes, the fixes, and SGD at scale.

Part B: the problems

Five mathematical and five real-life problems, each followed by a full worked solution.

Where we are going



1–4 how GD works

5–7 when it works, and when it fails

8–9 fixes, and scale

PART A

The ideas

The update is $\text{new value} = \text{old value} - \text{slope} \times \text{learning rate}$.

01

Why gradient descent?



Why GD

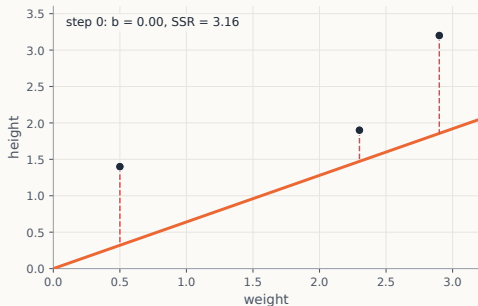
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

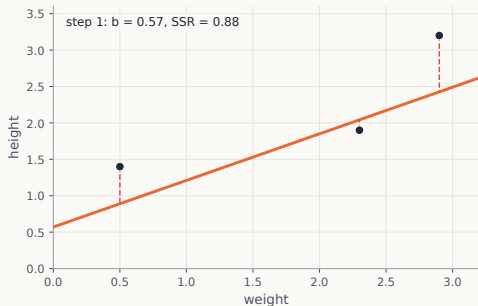
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

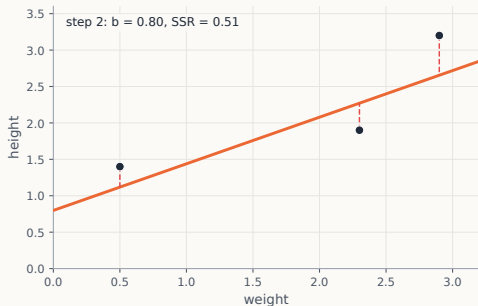
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

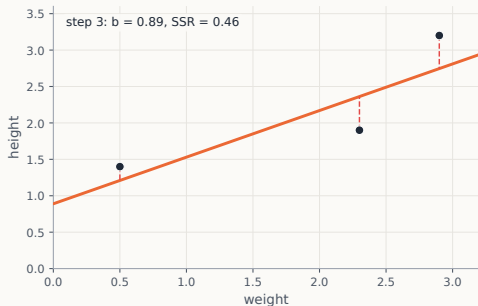
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

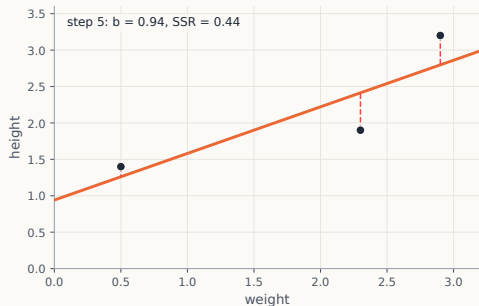
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

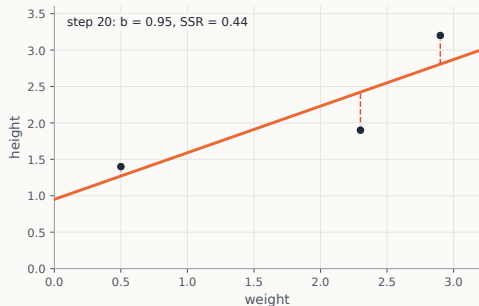
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.

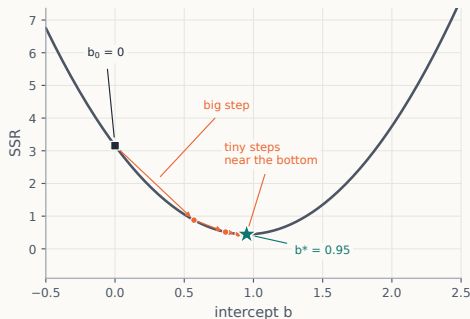


Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

At the best value the loss is flat, so solve $\text{loss}' = 0$. For a line we *can* (least squares). But often we **cannot**: there is no closed form (logistic regression, neural nets), there are millions of parameters, or new data keeps arriving.

The idea of gradient descent

Start from a guess and step downhill until the loss stops improving: **big steps** far away, **small steps** near the optimum. Cauchy proposed it in 1847 [6]; it is still used to train large models [3].



SSR against the intercept (slope 0.64). Steps shrink as the slope flattens.

02

The gradient: direction of steepest slope



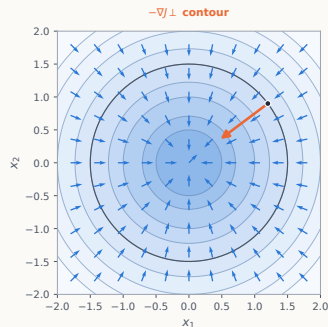
One variable: $J'(x)$ is the slope. Several: stack the partial derivative for **each** variable,

$$\nabla J(\mathbf{x}) = \begin{pmatrix} \partial J / \partial x_1 \\ \vdots \\ \partial J / \partial x_n \end{pmatrix}.$$

Example: $J = x_1^2 + x_2^2$ gives $\nabla J = (2x_1, 2x_2)$.

What the gradient tells us

At every point, ∇J is a vector that gives the **direction** and the **size** of the steepest slope. Moving along $-\nabla J$ goes downhill fastest.



Arrows: $-\nabla J$ on $J = x_1^2 + x_2^2$, pointing towards the minimum.

Why $-\nabla J$ is the steepest way down

The gradient: direction of steepest slope

Take a small step t in a unit direction $\mathbf{d}$. By Taylor, $J(\mathbf{x} + t\mathbf{d}) \approx J(\mathbf{x}) + t \nabla J(\mathbf{x})^\top \mathbf{d}$, so the rate of change along $\mathbf{d}$ is $\nabla J^\top \mathbf{d}$.

Claim

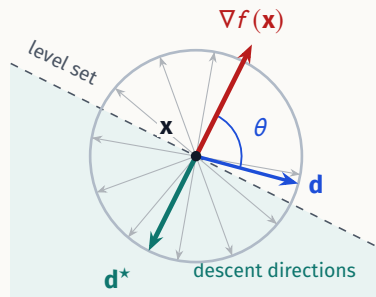
Among all unit directions, $\nabla J^\top \mathbf{d}$ is smallest at $\mathbf{d}^\star = -\nabla J / \|\nabla J\|$, where it equals $-\|\nabla J\|$.

Proof. By Cauchy–Schwarz,

$|\nabla J^\top \mathbf{d}| \leq \|\nabla J\| \|\mathbf{d}\| = \|\nabla J\|$, so $\nabla J^\top \mathbf{d} \geq -\|\nabla J\|$.

Equality holds when $\mathbf{d}$ parallel to ∇J , pointing the opposite way. ■

This identifies the direction of steepest descent [4, 6].



The rate along $\mathbf{d}$ is $\|\nabla J\| \cos \theta$, lowest at $\theta = 180^\circ$.

Contour plots, and why GD crosses them at right angles

A contour plot draws $J(x_1, x_2)$ as rings of equal height.

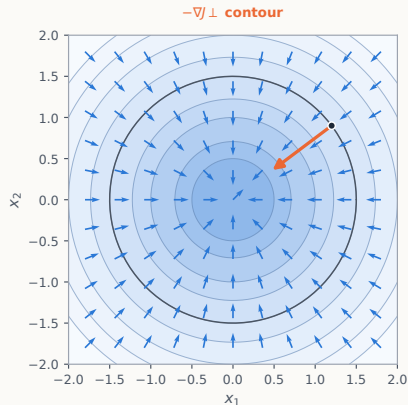
Why $\nabla J \perp$ contour. Move along a contour by a small $\mathbf{t}$; the height does not change:

$$J(\mathbf{x} + \mathbf{t}) - J(\mathbf{x}) \approx \nabla J^T \mathbf{t} = 0.$$

So each GD step leaves its contour at a right angle.

Reading a contour plot

Close contours: steeper slopes, larger steps.
Wide gaps: smaller steps.



Setting $\nabla J = 0$ finds *stationary* points. The second derivative tells them apart:

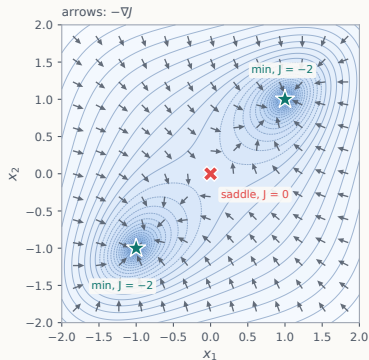
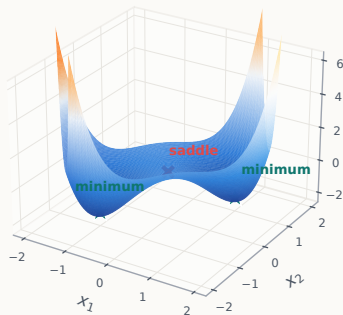
Function	Stationary point	Second derivative	Type
$J = x^2$	$J' = 2x = 0 \Rightarrow x = 0$	$J'' = 2 > 0$	minimum
$J = -x^2$	$x = 0$	$J'' = -2 < 0$	maximum
$J = x^3$	$J' = 3x^2 = 0 \Rightarrow x = 0$	$J'' = 6x = 0$, sign changes	saddle
$J = x_1^2 + x_2^2$	$\nabla J = (2x_1, 2x_2) = 0$	both curvatures $2 > 0$	minimum at $(0, 0)$

Zero gradient is not the same as a minimum

GD stops wherever the gradient vanishes, so it can stall at a saddle or settle in a *local* minimum. Problem M1 shows this on the quartic $J = x^4 - 6x^3 + 8x^2$.

What the three kinds of point look like

The gradient: direction of steepest slope



$J = x_1^4 + x_2^4 - 4x_1x_2$ has $\nabla J = 0$ at three points: two **minima** at $\pm(1, 1)$ and a **saddle** at the origin. The arrows $(-\nabla J)$ flow into the valleys, and at the saddle they arrive along one direction and leave along the other.

03

The update rule: $\text{step} = \text{slope} \times \text{learning rate}$



One parameter: the running example, with numbers

Fix the slope at $m = 0.64$ and let GD find the intercept b , starting from $b_0 = 0$ with learning rate $\alpha = 0.1$.

$$\frac{d \text{SSR}}{db} = -2 \sum_i (y_i - b - 0.64 x_i), \quad \text{step} = \frac{d \text{SSR}}{db} \times \alpha, \quad b_{\text{new}} = b_{\text{old}} - \text{step}.$$

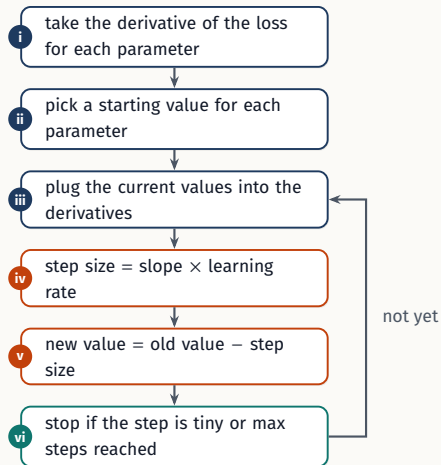
step k	b_k	slope $d\text{SSR}/db$	step size	$\text{SSR}(b_{k+1})$
0	0	-5.704	-0.5704	0.878
1	0.5704	-2.282	-0.2282	0.514
2	0.7986	-0.913	-0.0913	0.456
3	0.8898	-0.365	-0.0365	0.446

Each step is about $0.4\times$ the last (start $\text{SSR} = 3.156$), and b settles at $b^* = 0.951$.

Steps shrink because the slope flattens near the minimum.

Updating several parameters

The update rule: $\text{step} = \text{slope} \times \text{learning rate}$



For $\hat{y} = b + mx$ both parameters move:

$$\frac{\partial \text{SSR}}{\partial b} = -2 \sum_i (y_i - b - mx_i),$$

$$\frac{\partial \text{SSR}}{\partial m} = -2 \sum_i x_i (y_i - b - mx_i).$$

In general, for every w_j at once:

$$w_j \leftarrow w_j - \alpha \frac{\partial J}{\partial w_j}$$

Stop when the step size is near zero (say $< 10^{-3}$) or after a maximum number of steps.

Where the update comes from: first-order Taylor

Newton uses a *second*-order Taylor model. Use the *first*-order one, trusted only near $\mathbf{x}_k$ (constant $J(\mathbf{x}_k)$ dropped):

$$\mathbf{x}_{k+1} = \arg \min_{\mathbf{x}} \left\{ \underbrace{\nabla J(\mathbf{x}_k)^\top (\mathbf{x} - \mathbf{x}_k)}_{\text{straight-line model}} + \underbrace{\frac{1}{2\alpha} \|\mathbf{x} - \mathbf{x}_k\|^2}_{\text{stay close}} \right\}.$$

Setting the gradient of the braces to zero gives $\nabla J(\mathbf{x}_k) + \frac{1}{\alpha}(\mathbf{x} - \mathbf{x}_k) = 0$, i.e.

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla J(\mathbf{x}_k)$$

so the learning rate controls **how far we trust the straight-line model**.

	GD	Newton
Taylor model uses	1st order ∇J	2nd order ∇J , Hessian H
update	$\mathbf{x} - \alpha \nabla J$	$\mathbf{x} - H^{-1} \nabla J$
step length	set by α	set by curvature

Newton's method [7] offers another approach; Section 6 compares their costs.

Let a point slide downhill in continuous time. The **gradient flow** ODE always lowers J :

$$\dot{\mathbf{x}}(t) = -\nabla J(\mathbf{x}(t)) \implies \frac{d}{dt}J(\mathbf{x}(t)) = \nabla J^T \dot{\mathbf{x}} = -\|\nabla J\|^2 \leq 0.$$

Solve it with **forward Euler**, step h [7]: $\mathbf{x}_{k+1} = \mathbf{x}_k + h(-\nabla J(\mathbf{x}_k))$. That is **gradient descent with $\alpha = h$** , so Euler's stability analysis applies.

Numerical ODEs	Optimisation
Euler is stable only if $ 1 - h\lambda < 1$	the learning-rate limit $\alpha < 2/f''$ (Section 5)
stiff system (eigenvalues far apart)	stretched contours and the zig-zag (Section 4)
implicit (backward) Euler	the proximal-point method, stable for any step
a ball with mass and friction	momentum (Section 8)
$\ddot{\mathbf{x}} + \frac{3}{t}\dot{\mathbf{x}} + \nabla J = 0$	Nesterov's method [20]

THE ODE CONNECTION

**Gradient descent is forward Euler
on the gradient flow.**

So the learning-rate limit is a stability limit, and a stretched valley is a stiff ODE.

Many parameters at once: the vectorised update

Multiple linear regression [7]:

$\hat{y} = w_0 + w_1 x_1 + \dots + w_n x_n$ with the mean squared error

$$J(\mathbf{w}) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2.$$

Stack a column of ones and the features into $\mathbf{X}$. All partial derivatives come from one product,

$$\nabla J(\mathbf{w}) = \frac{1}{m} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}),$$

so one step costs $O(mn)$: two matrix-vector products, no matrix inversion.

```
import numpy as np

def gd(X, y, alpha=0.1, tol=1e-8,
       steps=10_000):
    X = np.column_stack([np.ones(len(y)), X])
    w = np.zeros(X.shape[1])          # (ii)
    for k in range(steps):
        g = X.T @ (X @ w - y) / len(y) # (iii)
        step = alpha * g                # (iv)
        w = w - step                    # (v)
        if np.linalg.norm(step) < tol:  # (vi)
            break
    return w, k
```

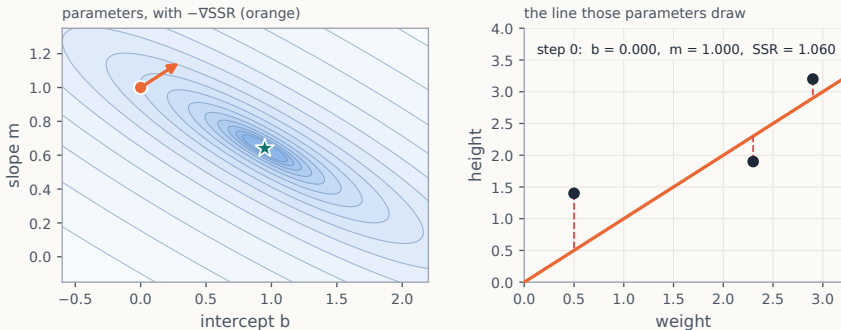
The comments match steps (i)–(vi) of the loop.

04

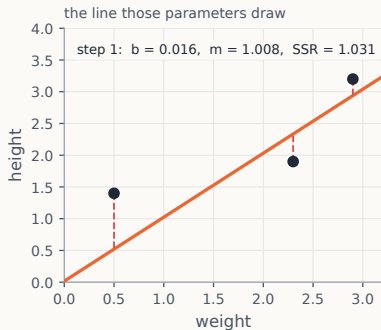
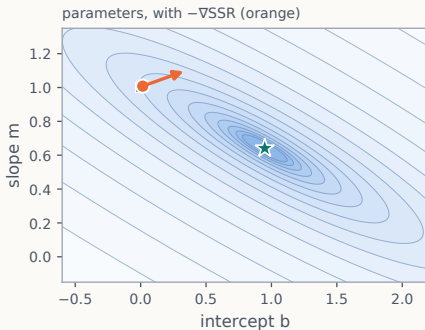
Watching it move



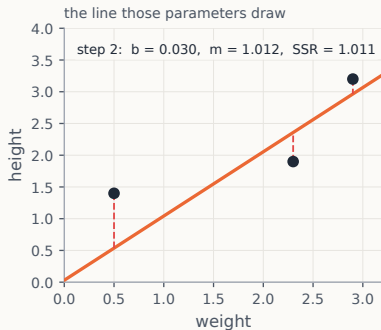
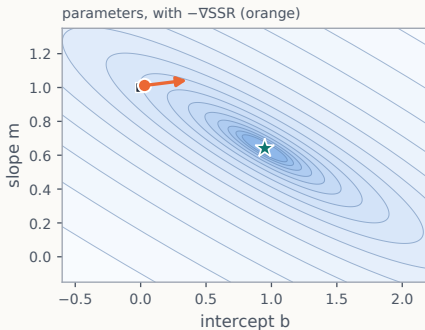
Watch it move



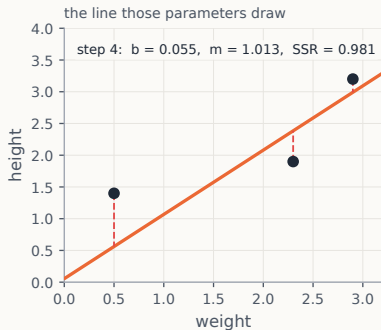
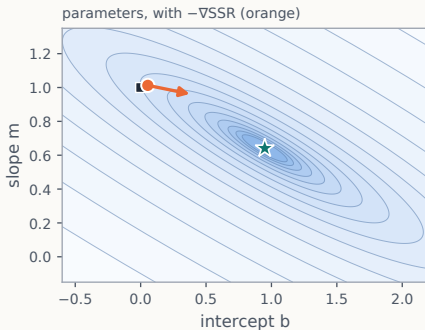
Weight–height data, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$, as in Problem A1. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



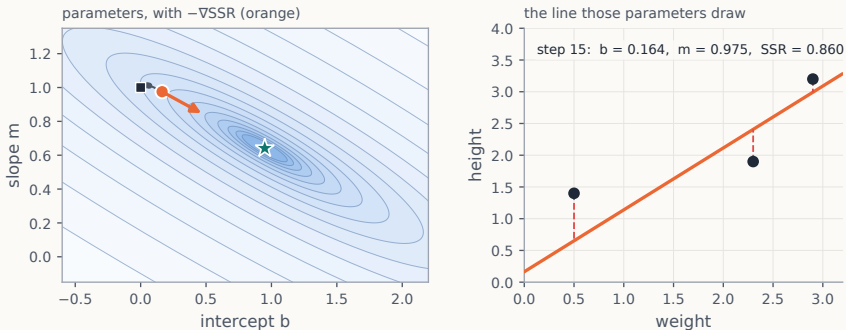
Weight–height data, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$, as in Problem A1. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



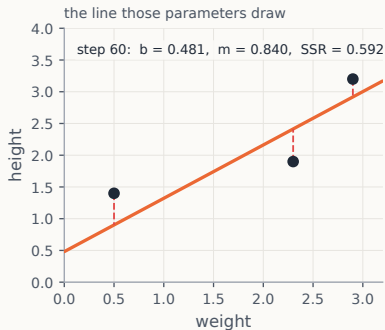
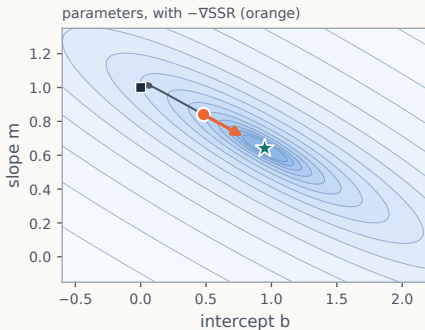
Weight–height data, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$, as in Problem A1. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



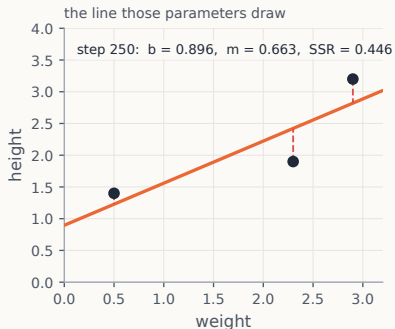
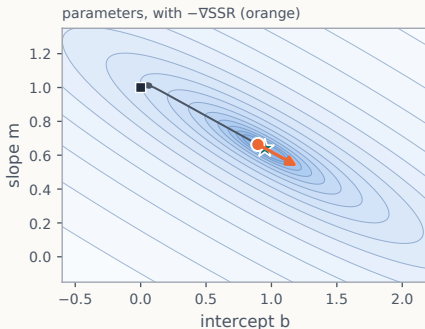
Weight–height data, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$, as in Problem A1. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



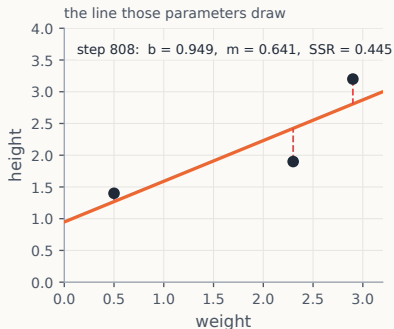
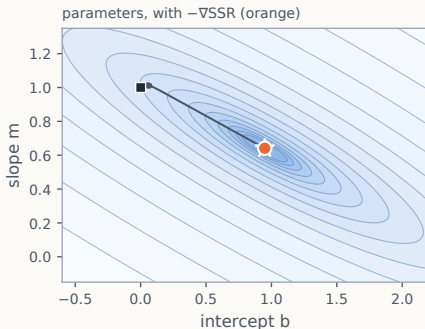
Weight–height data, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$, as in Problem A1. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



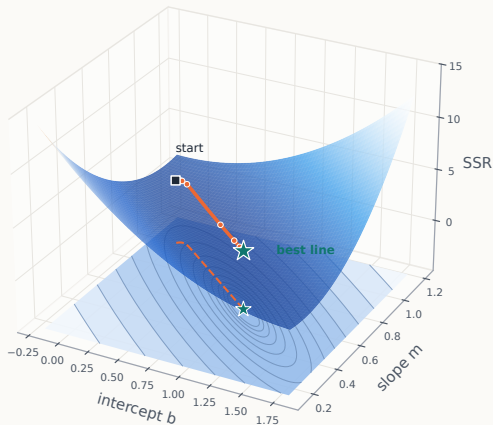
Weight–height data, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$, as in Problem A1. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



Weight–height data, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$, as in Problem A1. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.

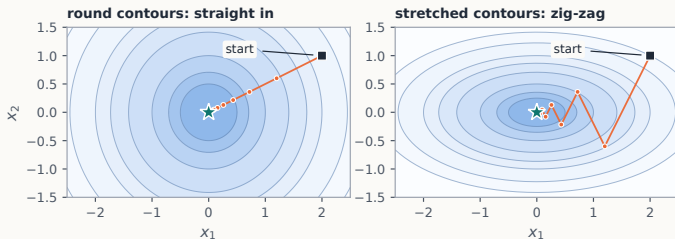


Weight–height data, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$, as in Problem A1. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



$SSR(b, m)$ is a bowl over the (b, m) plane. Every point of the bowl is one line through the data.

- GD starts at $(0, 1)$ and rolls downhill. The dashed curve is its shadow on the contour map, the path of the previous slide.
- It drops into the valley in a few steps, then creeps along the valley floor, because the valley is long and narrow ($\kappa \approx 29$, Section 7).
- After 808 steps it stops at $(0.9486, 0.6411)$, the least-squares line $(0.9487, 0.6410)$.

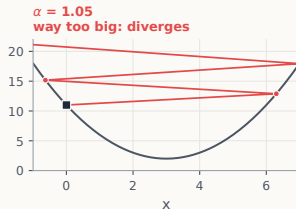
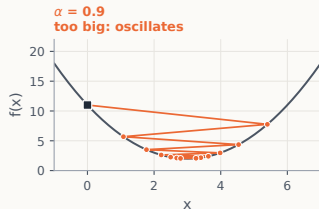
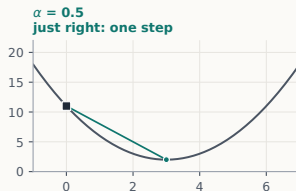
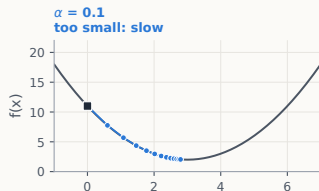


- Round rings ($x_1^2 + x_2^2$): $-\nabla J$ points straight at the minimum.
- Stretched rings ($x_1^2 + 4x_2^2$): it points across the valley; GD zig-zags. The steep direction ($f'' = 8$) caps $\alpha < 2/8$; the flat one ($f'' = 2$) then needs many steps.
- **Fix: scale the features** to similar curvature (standardise inputs, Problem A2).

05

How big a step? When GD works





GD on $f(x) = (x - 3)^2 + 2$ from $x_0 = 0$ with four learning rates:

- $\alpha = 0.1$: every step goes the right way, but slowly.
- $\alpha = 0.5$: lands on $x = 3$ in *one* step.
- $\alpha = 0.9$: jumps over the minimum each time, but still closes in.
- $\alpha = 1.05$: each jump is bigger than the last. It **diverges**.

Same function, same start: only α changed.

For $f(x) = (x - 3)^2 + 2$: $f'(x) = 2(x - 3)$. Write the error as $e_k = x_k - 3$:

$$x_{k+1} = x_k - 2\alpha(x_k - 3) \implies e_{k+1} = (1 - 2\alpha)e_k \implies e_k = (1 - 2\alpha)^k e_0.$$

learning rate	factor $1 - 2\alpha$	what happens
$0 < \alpha < 0.5$	between 0 and 1	steady approach from one side
$\alpha = 0.5$	0	exact in one step
$0.5 < \alpha < 1$	between -1 and 0	overshoots, oscillates inwards
$\alpha > 1$	below -1	diverges

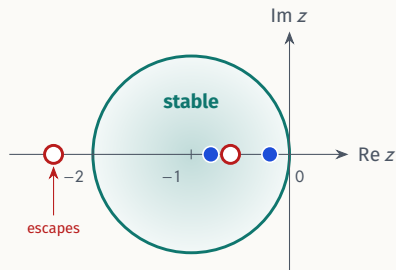
The general rule

With curvature $f'' > 0$ the error factor is $1 - \alpha f''$, so GD is stable only if $0 < \alpha < 2/f''$; best fixed step $\alpha = 1/f''$. Running example: $f'' = 6$, so $\alpha < 1/3$, and $\alpha = 0.1$ gives factor 0.4, the shrink in the table.

Euler's stability region explains the learning-rate limit

Take $J = x_1^2 + 4x_2^2$ (Problem M3). Its curvatures are $\lambda = 2$ and 8 , and each direction is the test equation $\dot{y} = -\lambda y$.

- The **exact flow** decays for every $\lambda > 0$: the continuous system is stable.
- **Forward Euler** multiplies y by $1 + z$ with $z = -\alpha\lambda$. It is stable only inside the disk $|1 + z| < 1$, i.e. $\alpha < 2/\lambda$ for every λ : $\alpha < 2/8$.
- This is **stiffness**: the steepest direction caps the step, the flattest one slows convergence.



- $\alpha = 0.1$: $z = -0.2, -0.8$, both inside.
- $\alpha = 0.3$: $z = -0.6, -2.4$, one is outside.

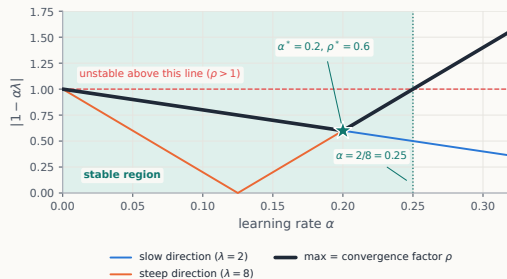
Theorem (quadratics) [17]

With curvatures $\lambda_{\min} \leq \lambda_i \leq \lambda_{\max}$, GD converges from every start iff $0 < \alpha < 2/\lambda_{\max}$, with rate

$$\rho(\alpha) = \max\{|1 - \alpha\lambda_{\min}|, |1 - \alpha\lambda_{\max}|\}.$$

The best step is $\alpha^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}$, giving $\rho^* = \frac{\kappa - 1}{\kappa + 1}$, $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$.

Proof. $|1 - \alpha\lambda|$ is largest at an end of $[\lambda_{\min}, \lambda_{\max}]$; the best α makes the two ends equal,
 $1 - \alpha\lambda_{\min} = \alpha\lambda_{\max} - 1$. ■



For M3: $\alpha^* = 2/10 = 0.2$ and $\rho^* = 3/5 = 0.6$. Close to a minimum every smooth J looks like a quadratic, so this describes GD near the minimum.

THE LEARNING RATE

**The learning rate controls
whether GD converges.**

Below $2/f''$ it converges. Above it, the error grows.

06

How fast does it converge?



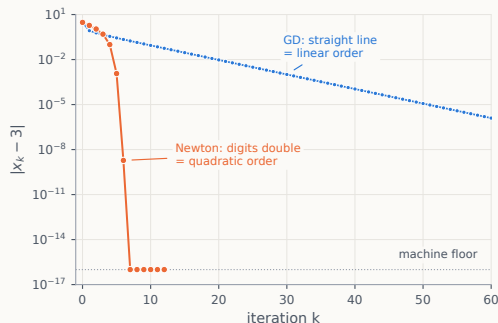
Near a minimum GD behaves like $e_{k+1} \approx \rho e_k$ with $\rho = |1 - \alpha f''|$: the error shrinks by the **same factor** every step. That is **linear** convergence (order $\rho = 1$).

To cut the error by 10^6 we need $\rho^k \leq 10^{-6}$:

$$k \geq \frac{\ln 10^6}{\ln(1/\rho)}.$$

- $f = (x - 3)^2$, $\alpha = 0.1$: $\rho = 0.8$, so $k = \boxed{62}$ steps.
- running example, $\alpha = 0.1$: $\rho = 0.4$, so $k = \boxed{16}$ steps.

Newton is **quadratic** ($\rho = 2$): correct digits roughly double each step.



Log scale: linear falls steadily; quadratic falls faster

When GD, and when Newton? Count the work for n parameters [17].

	Gradient descent	Newton's method
information used	gradient (n numbers)	gradient + Hessian (n^2 numbers)
cost of one step	$O(n)$ after the gradient	$O(n^3)$ to solve $H\mathbf{y} = \mathbf{g}$
order of convergence	linear	quadratic
learning rate	must be chosen	not needed
on a concave stretch ($f'' < 0$)	still goes downhill	can head for a maximum
best for	large problems (ML, $n \sim 10^6$)	small problems needing high accuracy

Why ML uses GD

With $n = 10^6$ a Hessian has 10^{12} entries and will not fit in memory; GD stores n numbers. When H is not positive definite, Levenberg–Marquardt [17] moves towards a gradient step.

FIRST VS. SECOND ORDER

First-order Taylor gives GD.
Second-order Taylor gives Newton.

GD needs more steps; Newton does more work per step.

Two assumptions: smooth, and strongly convex

How fast does it converge?

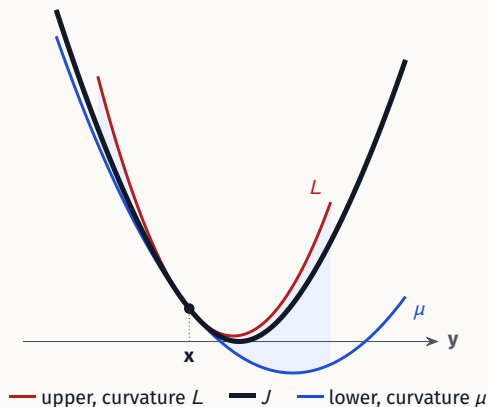
L -smooth: the slope cannot change too fast,
 $\|\nabla J(\mathbf{x}) - \nabla J(\mathbf{y})\| \leq L \|\mathbf{x} - \mathbf{y}\|$. In 1-D: $|J''| \leq L$.

μ -strongly convex: the bowl is curved at least by μ
everywhere. In 1-D: $J'' \geq \mu > 0$.

Together they squeeze J between two parabolas
through the current point. The **condition number**

$$\kappa = L/\mu \geq 1$$

measures how stretched the bowl is. It appears in
every rate that follows [15, 4].



Descent lemma [15]

If J is L -smooth, then for all $\mathbf{x}, \mathbf{y}$: $J(\mathbf{y}) \leq J(\mathbf{x}) + \nabla J(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$.

Proof idea. Write $J(\mathbf{y}) - J(\mathbf{x})$ as $\int_0^1 \nabla J(\mathbf{x} + t(\mathbf{y} - \mathbf{x}))^\top (\mathbf{y} - \mathbf{x}) dt$ and bound the change in the gradient by $L t \|\mathbf{y} - \mathbf{x}\|$. ■

Put in one GD step, $\mathbf{y} = \mathbf{x}_k - \alpha \mathbf{g}_k$:

$$J(\mathbf{x}_{k+1}) \leq J(\mathbf{x}_k) - \alpha \left(1 - \frac{\alpha L}{2}\right) \|\mathbf{g}_k\|^2 \stackrel{\alpha=1/L}{=} J(\mathbf{x}_k) - \frac{1}{2L} \|\mathbf{g}_k\|^2.$$

What it tells us

Any $\alpha < 2/L$ is *guaranteed* to lower J , the same window as in Section 5, now for any smooth J . Each step removes at least $\|\mathbf{g}_k\|^2 / 2L$, and every rate below is built on that drop [5].

Theorem (L -smooth, μ -strongly convex, $\alpha = 1/L$)

$$\|\mathbf{x}_k - \mathbf{x}^\star\|^2 \leq \left(1 - \frac{1}{\kappa}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2.$$

Proof. Let $\mathbf{e} = \mathbf{x}_k - \mathbf{x}^\star$ and $\Delta = J(\mathbf{x}_k) - J^\star$. Expand one step:

$\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 = \|\mathbf{e}\|^2 - \frac{2}{L} \mathbf{g}_k^\top \mathbf{e} + \frac{1}{L^2} \|\mathbf{g}_k\|^2$. Two facts:

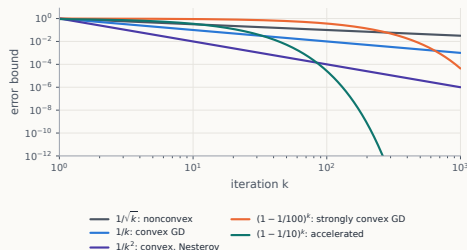
- strong convexity gives $\mathbf{g}_k^\top \mathbf{e} \geq \Delta + \frac{\mu}{2} \|\mathbf{e}\|^2$;
- the descent lemma gives $J^\star \leq J(\mathbf{x}_k) - \frac{1}{2L} \|\mathbf{g}_k\|^2$, so $\|\mathbf{g}_k\|^2 \leq 2L\Delta$.

$$\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 \leq \|\mathbf{e}\|^2 - \frac{2}{L} \Delta - \frac{\mu}{L} \|\mathbf{e}\|^2 + \frac{2}{L} \Delta = \left(1 - \frac{1}{\kappa}\right) \|\mathbf{e}\|^2. \quad \blacksquare$$

This is the *linear* order from the start of this section, now proved for every smooth, strongly convex function. It needs $O(\kappa \log \frac{1}{\epsilon})$ steps.

Assumption	Guarantee ($\alpha = \frac{1}{L}$)	Steps to ε
smooth only	$\min_k \ \mathbf{g}_k\ ^2 \leq \frac{2L\Delta_0}{K}$	$O(\varepsilon^{-2})$
convex	$J - J^\star \leq \frac{LR^2}{2k}$	$O(\varepsilon^{-1})$
strongly convex	$\ \mathbf{e}_k\ ^2 \leq (1 - \frac{1}{\kappa})^k R^2$	$O(\kappa \log \frac{1}{\varepsilon})$
PL [11]	$J - J^\star \leq (1 - \frac{1}{\kappa})^k \Delta_0$	$O(\kappa \log \frac{1}{\varepsilon})$

- Smooth only (deep learning): GD reaches a point with a small gradient, possibly a saddle.
- PL, $\frac{1}{2} \|\nabla J\|^2 \geq \mu(J - J^\star)$, needs no convexity and still gives a linear rate.
- $R = \|\mathbf{x}_0 - \mathbf{x}^\star\|$, $\Delta_0 = J(\mathbf{x}_0) - J^\star$.



07

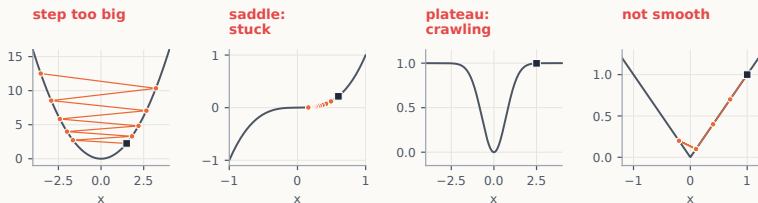
When and why GD fails



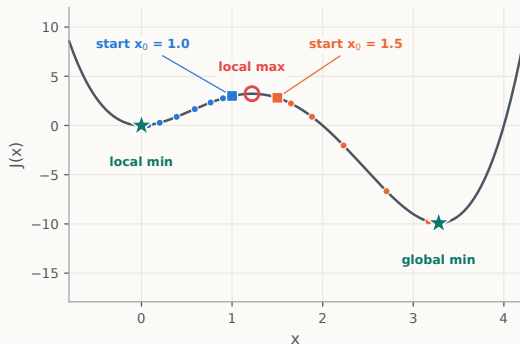
$\kappa = \lambda_{\max}/\lambda_{\min}$ of the Hessian indicates how slowly GD may converge.

Problem	κ	Why
M3: $x_1^2 + 4x_2^2$	4	curvatures 2 and 8
A1: line fit, raw weights	28.7	columns $[1, x]$ point in similar directions
A1: after centring x	1.04	centring removes the overlap
A2: houses, raw features	4.3×10^7	sizes in the hundreds, bedrooms in units
A2: after standardising	14.4	same scale; size and bedrooms still move together
Rosenbrock valley at $(1, 1)$	2508	a narrow curved valley

GD needs about κ steps per digit of accuracy, so preprocessing that reduces κ can speed up convergence.



- **Step too big:** $\alpha > 2/f''$, each step overshoots further (Section 5).
- **Saddle:** on x^3 the slope $3x^2 \rightarrow 0$, so GD stalls at a point that is not a minimum. Saddles with some negative curvature are escaped almost surely from a random start [13].
- **Plateau:** the gradient is nearly zero, so GD barely moves.
- **Not smooth:** $|x|$ has no derivative at 0; a fixed step keeps jumping across it. GD needs a smooth loss.



$J(x) = x^4 - 6x^3 + 8x^2$ has two valleys with a hill at $x \approx 1.22$.

- From $x_0 = 1.0$ GD rolls left into the *local* minimum $x = 0$.
- From $x_0 = 1.5$ it rolls right into the *global* minimum $x \approx 3.28$.

GD may find *a* minimum and miss *the* best one.

In practice

Try several starts and keep the best. The SSR of a line is convex: one minimum only.

08

Making it work better



Schedule [3]: start large, then shrink α , e.g.

$$\alpha_k = \frac{\alpha_0}{1 + \gamma k}.$$

Smaller late steps help SGD settle (Section 9).

Backtracking line search [1]

Try a step; if the loss did not drop enough, halve α and try again:

$$\text{accept } \alpha \text{ if } J(\mathbf{x} - \alpha \mathbf{g}) \leq J(\mathbf{x}) - c \alpha \|\mathbf{g}\|^2.$$

This picks a safe step automatically, without knowing f'' .

Rules of thumb

- If the loss goes *up*, α is too big: cut it by a factor of 10.
- If the loss falls very slowly in a straight line, α is too small.
- Always keep a maximum number of steps.
- Scale the features first (Section 4): it widens the range of good α .

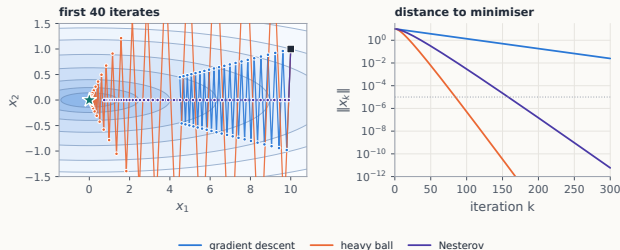
Polyak's heavy ball [18] adds a fraction β of the last move:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla J(\mathbf{x}_k) + \beta (\mathbf{x}_k - \mathbf{x}_{k-1}), \quad 0 \leq \beta < 1.$$

Where it comes from: a ball of mass m with friction γ obeys $m\ddot{\mathbf{x}} + \gamma\dot{\mathbf{x}} = -\nabla J(\mathbf{x})$. Replace the derivatives by finite differences with step h and solve for $\mathbf{x}_{k+1}$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \underbrace{\frac{h^2}{m+\gamma h}}_{\alpha} \nabla J(\mathbf{x}_k) + \underbrace{\frac{m}{m+\gamma h}}_{\beta} (\mathbf{x}_k - \mathbf{x}_{k-1}).$$

- Along the valley the gradient keeps its sign, so the velocity **builds up**. Across it the sign flips, so the moves **cancel**: the zig-zag is damped.
- Plain GD is the massless ball, $\beta = 0$: the gradient-flow ODE of Section 3.



On a quadratic, with the best α and β , the error factor per step is

$$\rho_{\text{HB}} = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \quad \text{vs.} \quad \rho_{\text{GD}} = \frac{\kappa-1}{\kappa+1}.$$

- $J = \frac{1}{2}(x_1^2 + 100x_2^2)$, $\kappa = 100$: 0.818 against 0.980, about 69 steps instead of 691 for six digits. In the run: GD 691, heavy ball 85, Nesterov 158.
- For nonquadratic losses, heavy ball can fail [14]. For illustrations, see [9].

Take the gradient at a **look-ahead point** [16]:

$$\mathbf{y}_k = \mathbf{x}_k + \beta_k(\mathbf{x}_k - \mathbf{x}_{k-1}), \quad \mathbf{x}_{k+1} = \mathbf{y}_k - \frac{1}{L} \nabla J(\mathbf{y}_k).$$

This gives guarantees for *every* smooth convex J , not just quadratics, and no first-order method can beat them by more than a constant [15]:

	GD	Nesterov	best possible
convex, error after k steps	$O(1/k)$	$O(1/k^2)$	$\Omega(1/k^2)$
strongly convex, steps to ε	$O(\kappa \log \frac{1}{\varepsilon})$	$O(\sqrt{\kappa} \log \frac{1}{\varepsilon})$	$\Omega(\sqrt{\kappa} \log \frac{1}{\varepsilon})$
J goes down every step?	yes	no (it ripples)	

Where GD needs 10^4 steps, Nesterov needs about 200. Its continuous-time limit is $\ddot{\mathbf{x}} + \frac{3}{t} \dot{\mathbf{x}} + \nabla J = 0$, a heavy ball whose friction fades [20].

CONDITIONING AND SPEED

**The condition number κ
controls the iteration count.**

GD needs about κ . Momentum and Nesterov need about $\sqrt{\kappa}$, and these rates are optimal for first-order methods.

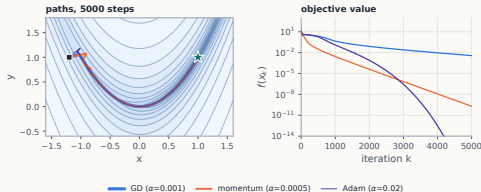
Adam [12], all operations element-wise:

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k$$

$$\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \hat{\mathbf{m}}_k / (\sqrt{\hat{\mathbf{v}}_k} + \epsilon)$$

- **m** is **momentum**, as above.
- Dividing by $\sqrt{\mathbf{v}}$ gives each parameter its **own learning rate**: automatic feature scaling (Section 4).
- The hats correct the start-up bias. Adam is a common deep learning optimiser.



Rosenbrock, 5000 steps: GD is slow ($J \approx 4 \times 10^{-3}$), heavy ball reaches 2×10^{-10} , Adam reaches (1, 1).

09

At scale: SGD and modern practice

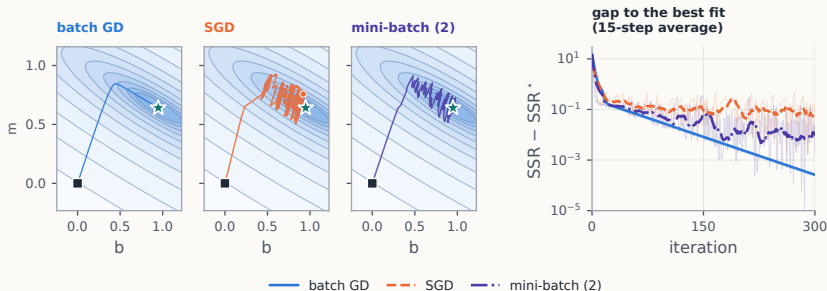


Plain (batch) GD uses *every* data point to compute each gradient. With millions of points and parameters, one step is already expensive.

Variant	data used per step	character
batch GD	all m points	smooth path, expensive steps
stochastic GD (strict)	one random point	cheap, very noisy steps
mini-batch GD (common practice)	a small random subset	cheap and fairly stable

Why use mini-batches [3]

Mini-batches give more stable estimates than one point, while keeping steps cheap. When **new data** arrives, updates can continue from the current parameters.



- One-point gradients are *unbiased*: on average they equal the full gradient.
- Their noise does not vanish at the minimum, so with a fixed α SGD keeps jittering. Averaging B points cuts the variance by B ; a decaying α lets the path settle.

One-sample gradients are right *on average*, but at the minimum they are not zero:

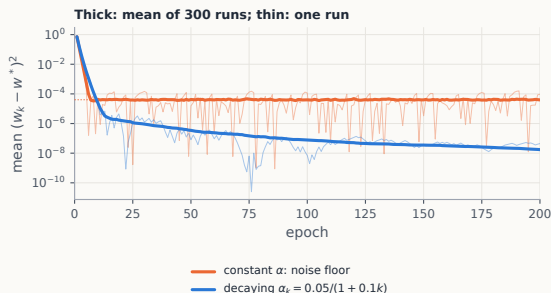
$\nabla J(w^*) = 0$ while each $\nabla J_i(w^*) \neq 0$.

- Constant α : fast at first, then limited by a **noise floor** that grows with α .
- Decaying α_k : the noise floor falls, at rate $O(1/k)$ for strongly convex J [3].

Robbins-Monro [19]: SGD converges if

$$\sum_k \alpha_k = \infty, \quad \sum_k \alpha_k^2 < \infty,$$

steps persist while their squared sizes remain summable.



The sensor data of Problem A3, averaged over 300 shuffled runs. After 200 epochs: constant α sits near 4×10^{-5} , the schedule reaches 2×10^{-8} .

Where the gradient comes from: automatic differentiation

Cheap gradient principle [10]

Reverse-mode automatic differentiation computes the *whole* gradient $\nabla J \in \mathbb{R}^n$ for a small constant times the cost of evaluating J once, **however large n is**.

How we get ∇J	cost	accuracy
by hand (as in Part B)	free once derived	exact
finite differences, $\frac{J(\mathbf{x}+h\mathbf{e}_i) - J(\mathbf{x})}{h}$	$(n+1) \times$ cost of J	about $\sqrt{\epsilon_{\text{mach}}}$
reverse-mode AD (backpropagation)	a few times the cost of J	machine precision

Backpropagation in neural networks is reverse-mode AD. That, plus GD's $O(n)$ update, is why a model with 10^9 parameters can be trained at all.

Barzilai–Borwein: learn α from the last move [2]

$$\alpha_k = \frac{\mathbf{s}^\top \mathbf{s}}{\mathbf{s}^\top \mathbf{r}}, \quad \mathbf{s} = \mathbf{x}_k - \mathbf{x}_{k-1}, \quad \mathbf{r} = \mathbf{g}_k - \mathbf{g}_{k-1}.$$

- $\mathbf{r} \approx H\mathbf{s}$, so α_k estimates $1/f''$ along the last step: Newton's step size, from gradients only.
- J may briefly go *up*, yet it is often faster than a fixed α . Pair it with Armijo backtracking [1] for safety.

The edge of stability [8]

- Theory says keep $\alpha < 2/L$.
- Yet when full-batch GD trains a neural network, the sharpness $\lambda_{\max}$ *rises* until it reaches $2/\alpha$, then hovers there while the loss keeps falling, unevenly.
- The analysis of Sections 5 and 6 does not explain this. It is an open research question.

10

Summary



recap

Idea	In one line
update	$w_j \leftarrow w_j - \alpha \partial J / \partial w_j$ (step size = slope $\times$ learning rate)
gradient	direction and size of the steepest slope; $-\nabla J$ is the steepest way down
stop	step size ≈ 0 , or maximum number of steps reached
stability	converges only if $0 < \alpha < 2/f''$; best fixed $\alpha = 1/f''$
speed	linear order: error $\times 1 - \alpha f'' $ per step; Newton is quadratic
cost	$O(mn)$ per step, no matrix inversion: the reason ML uses it
geometry	steps cross contours at right angles; stretched contours make it zig-zag
failures	too-big α , saddles, plateaus, non-smooth losses, local minima
SGD	one point (or a mini-batch) per step: cheap and noisy; decay α to settle

Idea	In one line
ODE view	GD is forward Euler on $\dot{\mathbf{x}} = -\nabla J$; $\alpha < 2/\lambda_{\max}$ is Euler's stability limit
best step	on a quadratic $\alpha^* = 2/(\lambda_{\min} + \lambda_{\max})$, error factor $(\kappa - 1)/(\kappa + 1)$
guarantees	descent lemma: each step drops J by $\ \mathbf{g}\ ^2/2L$; strongly convex gives $(1 - 1/\kappa)^k$
condition number	$\kappa = L/\mu$ means about κ steps per digit; scaling features shrinks it
momentum	heavy ball and Nesterov need about $\sqrt{\kappa}$, and no gradient method can do better
SGD theory	a constant α leaves a noise floor; Robbins–Monro schedules remove it
practice	Adam = momentum + per-parameter steps; backpropagation makes ∇J cheap

IN ONE SENTENCE

**Take a step downhill,
sized by the slope and the learning rate.**

Choose the learning rate carefully and check where the method converges.

B

Part B: problems and complete solutions



Each problem is followed **right away** by a worked solution. Try it before turning the page. Every number in the solutions was checked by a script.

Mathematical

- M1** Stationary points and local minima of a quartic
- M2** Four learning rates on one parabola
- M3** Two variables, contours and the zig-zag
- M4** Scaling and shifting the loss
- M5** GD against Newton

Real-life applications

- A1** Health: weight vs. height, the running example
- A2** Real estate: house prices with two features
- A3** Sensors: streaming data with SGD
- A4** Engineering: the cheapest drink can
- A5** Education: predicting pass or fail

Difficulty: ★ routine ★★ several steps ★★★ deeper idea.

Problem M1

Mathematical ★★

Let $J(x) = x^4 - 6x^3 + 8x^2$.

- (a) Find every stationary point of J .
- (b) Use J'' to say which are minima and which are maxima. Which minimum is the lowest?
- (c) Run three GD steps with $\alpha = 0.05$ from $x_0 = 1.0$, and again from $x_0 = 1.5$. Where does each run end up?
- (d) How reliable is the result from a single GD run?

Practice: $J' = 0$, the second-derivative test, and the effect of the starting point.

(a) **Stationary points.** Set the slope to zero and factor out $2x$:

$$J'(x) = 4x^3 - 18x^2 + 16x = 2x(2x^2 - 9x + 8) = 0.$$

So $x = 0$, or $x = \frac{9 \pm \sqrt{81 - 64}}{4} = \frac{9 \pm \sqrt{17}}{4}$, which gives

$$x = 0, \quad x \approx 1.2192, \quad x \approx 3.2808$$

(b) **Classify** with $J''(x) = 12x^2 - 36x + 16$:

x	$J''(x)$	$J(x)$	type
0	$16 > 0$	0	local minimum
1.2192	$-10.05 < 0$	3.227	maximum (the hill)
3.2808	$27.05 > 0$	-9.915	global minimum

(c) GD with $\alpha = 0.05$, $x_{k+1} = x_k - 0.05 J'(x_k)$.

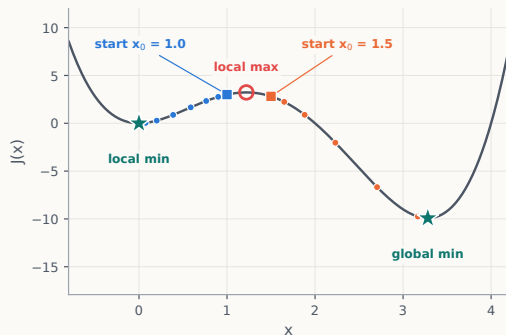
From $x_0 = 1.0$: $J'(1) = 4 - 18 + 16 = 2$, so
 $x_1 = 1 - 0.1 = 0.9$.

$$1 \rightarrow 0.9 \rightarrow 0.763 \rightarrow 0.588 \rightarrow \boxed{0}$$

From $x_0 = 1.5$: $J'(1.5) = 13.5 - 40.5 + 24 = -3$,
 so $x_1 = 1.65$.

$$1.5 \rightarrow 1.65 \rightarrow 1.882 \rightarrow 2.231 \rightarrow \boxed{3.281}$$

The two starts sit on opposite sides of the hill at
 1.2192.



(d) What it means.

- GD only follows the local slope. Start left of the hill and it settles at $x = 0$ with $J = 0$. Start right of it and it reaches $x \approx 3.28$ with $J \approx -9.92$.
- Both runs *converged*: the step size went to zero in each. Convergence tells you that you reached a stationary point, not that you reached the best one.
- In practice, try several starting points and keep the lowest J .

Answer

Stationary points $0, (9 \pm \sqrt{17})/4$. The global minimum is $x^* \approx 3.2808, J^* \approx -9.915$. GD finds it only from starts to the right of the hill at $x \approx 1.2192$.

Problem M2 · Four learning rates on one parabola

Problem M2

Mathematical ★

Minimise $f(x) = (x - 3)^2 + 2$ with GD from $x_0 = 0$.

- (a) Write out $x_1, \dots, x_4$ for $\alpha = 0.1, 0.5, 0.9, 1.1$.
- (b) Show that the error $e_k = x_k - 3$ obeys $e_{k+1} = (1 - 2\alpha) e_k$.
- (c) Use (b) to explain each of the four behaviours.
- (d) With $\alpha = 0.1$, how many steps cut the error by a factor of 10^6 ?

Practice: running the update by hand, and the stability window for α .

(a) The iterates. Here $f'(x) = 2(x - 3)$, so $x_{k+1} = x_k - 2\alpha(x_k - 3)$.

α	x_1	x_2	x_3	x_4	behaviour
0.1	0.6	1.08	1.464	1.7712	creeps up from one side
0.5	3	3	3	3	exact in <i>one</i> step
0.9	5.4	1.08	4.536	1.7712	jumps across, closes in
1.1	6.6	-1.32	8.184	-3.2208	diverges

For example, with $\alpha = 0.1$: $x_1 = 0 - 0.2(0 - 3) = 0.6$, then $x_2 = 0.6 - 0.2(0.6 - 3) = 1.08$.

The figure in Section 5 shows the same four regimes, drawn with $\alpha = 1.05$ for the last one.

(b) The error recursion. Subtract 3 from both sides of the update:

$$x_{k+1} - 3 = (x_k - 3) - 2\alpha(x_k - 3) \implies e_{k+1} = (1 - 2\alpha)e_k, \quad e_k = (1 - 2\alpha)^k e_0.$$

(c) Reading the factor $\rho = 1 - 2\alpha$, with $e_0 = -3$:

- $\alpha = 0.1$: $\rho = 0.8$. Same sign every step, shrinking by 20%: a slow, one-sided approach.
- $\alpha = 0.5$: $\rho = 0$. The error vanishes after one step.
- $\alpha = 0.9$: $\rho = -0.8$. The sign flips each step but $|\rho| < 1$, so it still converges.
- $\alpha = 1.1$: $\rho = -1.2$. The sign flips and $|e_k|$ grows by 20% each step.

So GD converges exactly when $|1 - 2\alpha| < 1$, that is $0 < \alpha < 1$, which matches $2/f'' = 2/2$.

(d) **Counting steps for $\alpha = 0.1$.** We need $|e_k/e_0| = 0.8^k \leq 10^{-6}$:

$$k \geq \frac{\ln 10^6}{\ln(1/0.8)} = \frac{13.816}{0.2231} = 61.9 \implies \boxed{k = 62}$$

Check: $0.8^{61} = 1.23 \times 10^{-6}$ is not enough, while $0.8^{62} = 9.8 \times 10^{-7}$ is.

Take-away

With the function and start fixed, the outcome depends on α . The best fixed step is $\alpha = 1/f'' = 0.5$; anything past $2/f'' = 1$ diverges.

Problem M3 · Two variables, contours and the zig-zag

Problem M3

Mathematical ★★

Let $J(x_1, x_2) = x_1^2 + 4x_2^2$ and start from $\mathbf{x}_0 = (2, 1)$.

- (a) Find ∇J and take two GD steps with $\alpha = 0.1$. Does J go down?
- (b) What is the largest α for which GD still converges? Take two steps with $\alpha = 0.2$.
- (c) Repeat (a) on the round bowl $x_1^2 + x_2^2$. Compare the two paths.

Practice: the update with several parameters, and how the shape of the contours changes GD.

(a) Two steps with $\alpha = 0.1$. $\nabla J = (2x_1, 8x_2)$, so each coordinate has its own factor:

$$x_1 \leftarrow (1 - 2\alpha) x_1 = 0.8 x_1, \quad x_2 \leftarrow (1 - 8\alpha) x_2 = 0.2 x_2.$$

step	$\mathbf{x}_k$	$\nabla J(\mathbf{x}_k)$	$J(\mathbf{x}_k)$
0	(2, 1)	(4, 8)	8
1	(1.6, 0.2)	(3.2, 1.6)	2.72
2	(1.28, 0.04)	(2.56, 0.32)	1.6448

J falls from 8 to 1.64. The steep x_2 direction is almost done after two steps, but x_1 has only lost 36%: the flat direction is the slow one.

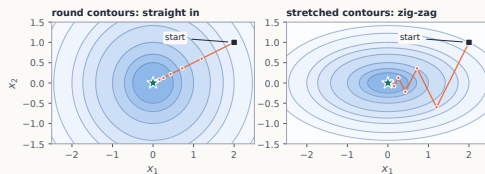
(b) Largest stable α . Both factors must satisfy $|1 - 2\alpha| < 1$ and $|1 - 8\alpha| < 1$. The steep direction is stricter:

$$0 < \alpha < \frac{2}{8} = \boxed{0.25}$$

With $\alpha = 0.2$ the factors are 0.6 and -0.6 :

$$(2, 1) \rightarrow (1.2, -0.6) \rightarrow (0.72, 0.36)$$

x_2 flips sign every step. That is the **zig-zag** across the valley. It still converges, since $0.6 < 1$.



(c) **The round bowl** $x_1^2 + x_2^2$. Now $\nabla J = (2x_1, 2x_2)$, and with $\alpha = 0.1$ both factors are 0.8:

$$(2, 1) \rightarrow (1.6, 0.8) \rightarrow (1.28, 0.64), \quad J : 5 \rightarrow 3.2 \rightarrow 2.048.$$

Every point is a multiple of $(2, 1)$: the path is a **straight line** into the minimum, because $-\nabla J$ points right at it.

Take-away

On the round bowl any $\alpha < 1$ works and $\alpha = 0.5$ finishes in one step. On the stretched bowl the steep direction caps α at 0.25, and the flat direction converges slowly. Equal curvatures speed up GD.

Problem M4 · SSR or MSE? Scaling and shifting the loss

Problem M4

Mathematical ★★

Use the clinic data $x = (0.5, 2.3, 2.9)$, $y = (1.4, 1.9, 3.2)$ with the slope fixed at 0.64. Compare two losses for the intercept b :

$$\text{SSR}(b) = \sum_{i=1}^3 (y_i - b - 0.64x_i)^2, \quad J(b) = \frac{1}{2m} \text{SSR}(b), \quad m = 3.$$

- (a) Find both gradients at $b = 0$ and the first GD step with $\alpha = 0.1$.
- (b) Do the two losses have the same minimiser? Find it.
- (c) What changes if we add the constant 2 to the loss?

(a) Gradients at $b = 0$. Write $r_i = y_i - 0.64x_i = (1.08, 0.428, 1.344)$, so $\sum r_i = 2.852$.

$$\frac{d \text{SSR}}{db} = -2 \sum_i (r_i - b) = -2(2.852) = -5.704, \quad \frac{dJ}{db} = \frac{1}{6} \frac{d \text{SSR}}{db} = -0.9507.$$

First step with $\alpha = 0.1$:

$$\text{SSR : } b_1 = 0 + 0.5704 = 0.5704, \quad J : b_1 = 0 + 0.09507 = 0.0951$$

Same direction, but the MSE step is $2m = 6$ times smaller. Dividing the loss by 6 is the same as dividing α by 6: $\alpha = 0.6$ on J takes exactly the SSR steps.

(b) Same minimiser. Setting either derivative to zero gives the same equation,

$$\sum_i (r_i - b) = 0:$$

$$b^* = \frac{1}{3} \sum_i r_i = \frac{2.852}{3} = 0.9507$$

Scaling a loss by a positive number cannot move its lowest point; both GD runs end at 0.950667.

(c) Adding 2. SSR(0) goes from 3.156 to 5.156, but the derivative of a constant is zero, so the gradient, every step and the minimiser are **unchanged**.

Take-away

GD uses the slope: shifting leaves updates unchanged; scaling rescales α . In $\frac{1}{2m}$, the $\frac{1}{2}$ cancels the 2 from the square and $\frac{1}{m}$ makes α independent of the data size.

Problem M5

Mathematical ★ ★ ★

- (a) Apply one Newton step, $x_{k+1} = x_k - f'(x_k)/f''(x_k)$, to $f(x) = (x - 3)^2$ from $x_0 = 0$. How many GD steps with $\alpha = 0.1$ reach the same accuracy (error cut by 10^6)?
- (b) Now take $J(x) = x^4 - 6x^3 + 8x^2$ from Problem M1 and start at $x_0 = 1.4$. Run Newton and GD ($\alpha = 0.05$). Where does each one go?
- (c) Which method would you trust here, and why?

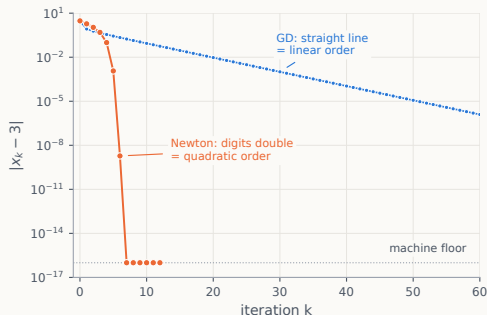
Practice: comparing GD and Newton with numerical examples.

(a) **Newton on a parabola.** $f'(0) = -6$ and $f'' = 2$:

$$x_1 = 0 - \frac{-6}{2} = \boxed{3}$$

Newton reaches the minimum. Newton fits a parabola to f , and here f is a parabola.

GD with $\alpha = 0.1$ cuts the error by 0.8 per step, so it needs $\boxed{62}$ steps (Problem M2). Newton's step is $\alpha = 1/f''$, the best possible fixed rate.



(b) Start at $x_0 = 1.4$ on the quartic. Here $J'(1.4) = -1.904$ and $J''(1.4) = -10.88 < 0$: the curve bends *down*.

Newton: $x_1 = 1.4 - \frac{-1.904}{-10.88} = 1.4 - 0.175 = 1.225$, then

$1.4 \rightarrow 1.225 \rightarrow 1.21923 \rightarrow \boxed{1.21922}$ the **maximum**, $J = 3.227$.

GD: $x_1 = 1.4 - 0.05(-1.904) = 1.4952$, then

$1.4 \rightarrow 1.4952 \rightarrow 1.6426 \rightarrow 1.8704 \rightarrow \dots \rightarrow \boxed{3.2808}$ the global minimum, $J = -9.915$.

Newton converged quickly to a maximum.

(c) Why. Newton solves $J'(x) = 0$. It cannot tell a minimum from a maximum, and where $J'' < 0$ its step points *uphill*. GD steps along $-J'$, which always points downhill, so J decreases for any small enough α .

from $x_0 = 1.4$	Newton	GD, $\alpha = 0.05$
limit	1.2192 (maximum)	3.2808 (minimum)
speed near the limit	quadratic	linear

Take-away

Trust GD far from the answer, and Newton close to a minimum where $J'' > 0$. Levenberg-Marquardt [17] uses both gradients and curvature.

Problem A1

Application ★

A clinic records weight x and height y for three people: $x = (0.5, 2.3, 2.9)$, $y = (1.4, 1.9, 3.2)$. Fit $\hat{y} = b + mx$ by minimising $\text{SSR}(b, m)$.

- (a) Fix $m = 0.64$. Run three GD steps on b from $b_0 = 0$ with $\alpha = 0.1$. When does the condition “stop when $|\text{step}| < 0.001$ ” hold?
- (b) Now move both: start at $(b, m) = (0, 1)$ with $\alpha = 0.01$ and take two steps.
- (c) Solve the least-squares equations by hand and compare with where GD ends.

Practice: steps (i)–(vi), worked by hand.

(a) **Intercept only.** $\frac{d \text{SSR}}{db} = -2 \sum_i (y_i - b - 0.64x_i)$, step = slope $\times 0.1$, $b \leftarrow b - \text{step}$.

step	slope $d\text{SSR}/db$	step size	new b
1	-5.704	-0.5704	0.5704
2	-2.2816	-0.22816	0.79856
3	-0.91264	-0.091264	0.889824

Each step is 0.4 times the one before, so it takes 8 updates for the step to drop below 0.001 (last step 0.00093). GD stops at $b = 0.9500$, just short of $b^* = 0.9507$.

(b) Both parameters. The residuals at $(b, m) = (0, 1)$ are $y_i - x_i = (0.9, -0.4, 0.3)$:

$$\frac{\partial \text{SSR}}{\partial b} = -2(0.9 - 0.4 + 0.3) = -1.6, \quad \frac{\partial \text{SSR}}{\partial m} = -2(0.45 - 0.92 + 0.87) = -0.8.$$

Both move *at the same time*, with $\alpha = 0.01$:

$$(b, m)_1 = (0 + 0.016, 1 + 0.008) = (0.016, 1.008)$$

At the new point the gradient is $(-1.4128, -0.3944)$, so

$$(b, m)_2 = (0.030128, 1.01194)$$

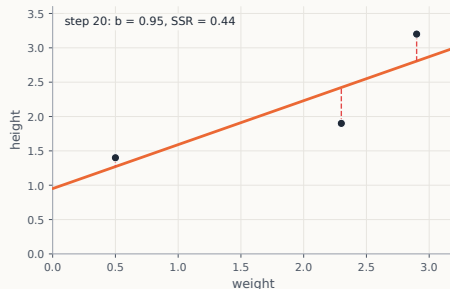
After 808 updates both steps are below 10^{-6} and $(b, m) = (0.94863, 0.64106)$.

(c) Least squares by hand. With $n = 3$, $\sum x = 5.7$, $\sum y = 6.5$, $\sum x^2 = 13.95$, $\sum xy = 14.35$:

$$m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2} = \frac{6.00}{9.36} = 0.641026$$

$$b = \frac{\sum y - m \sum x}{n} = 0.948718$$

GD matched both to within 10^{-4} . Here a closed form is easy; GD uses the gradient instead.



Problem A2 · Real estate: pricing houses with two features

Problem A2

Application ★ ★ ★

Four flats in Dhaka:

size (sq ft)	800	1000	1200	1500
bedrooms	2	2	3	3
price (lakh Tk)	40	50	60	70

Fit $\hat{y} = w_0 + w_1 \text{ size} + w_2 \text{ beds}$ with $J(\mathbf{w}) = \frac{1}{2m} \sum_i (\hat{y}_i - y_i)^2$.

- (a) Find ∇J at $\mathbf{w} = 0$ on the raw data. Why is plain GD slow here?
- (b) Standardise both features and run GD with $\alpha = 0.1$. Map the weights back to original units.
- (c) Check with the normal equation, then price a 1100 sq ft, 2-bedroom flat.

(a) **The raw gradient.** At $\mathbf{w} = 0$, $\nabla J = \frac{1}{m} \mathbf{X}^T (\mathbf{X} \mathbf{w} - \mathbf{y}) = -\frac{1}{m} \mathbf{X}^T \mathbf{y}$:

$$\nabla J(0) = (-55, -64\,750, -142.5).$$

The size component is hundreds of times bigger than the others, because sizes are in the hundreds.

- Stability needs $\alpha < 2/\lambda_{\max}$, where $\lambda_{\max} = 1.33 \times 10^6$ is the largest curvature. So $\alpha < 1.5 \times 10^{-6}$.
- With such a tiny α , the step for w_0 is about $10^{-6} \times 55$: the intercept barely moves.

This is the stretched-contour zig-zag of Section 4, with very unequal scales.

(b) Standardise, $z = (\text{value} - \text{mean})/\text{std}$: size mean 1125, std 258.60; beds mean 2.5, std 0.5.

Now every feature has a similar scale, and GD with $\alpha = 0.1$ converges in 1391 steps (until $\|\nabla J\| < 10^{-8}$) to

$$\hat{y} = 55 + 9.946 z_{\text{size}} + 1.346 z_{\text{beds}}.$$

Back to original units: divide each weight by its std, then fix the intercept.

$$w_1 = \frac{9.946}{258.60} = 0.03846, \quad w_2 = \frac{1.346}{0.5} = 2.692,$$

$$w_0 = 55 - 0.03846(1125) - 2.692(2.5) = 5.000.$$

(c) Normal equation check. Solving $\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y}$ directly gives

$$\mathbf{w} = (5, 0.038462, 2.692308),$$

the same as GD to within 10^{-7} . So the model is

$$\text{price} \approx 5 + 0.0385 \times \text{size} + 2.69 \times \text{beds} \quad (\text{lakh Tk}).$$

For 1100 sq ft and 2 bedrooms: $5 + 42.31 + 5.38 = 52.69$ lakh Tk.

Take-away

A large feature scale slows raw GD; standardising the same data speeds convergence. Each extra square foot adds about Tk 3,850, and each extra bedroom about Tk 2.7 lakh.

Problem A3

Application ★★

A sensor gives readings y for known inputs x : $(1, 2.1)$, $(2, 3.9)$, $(3, 6.2)$. Calibrate $\hat{y} = wx$ with the per-sample loss $\frac{1}{2}(wx_i - y_i)^2$, $w_0 = 0$, $\alpha = 0.05$.

- (a) Run one SGD epoch (samples in order 1, 2, 3).
- (b) Compare with one batch GD step, and with mini-batches $\{1, 2\}$ then $\{3\}$.
- (c) Find the exact w^* . After 50 epochs, compare a constant α with $\alpha_k = 0.05/(1 + 0.1k)$.
- (d) A new reading $(4, 8.1)$ arrives. Update the model without starting again.

(a) One SGD epoch. For one sample the gradient is $(w x_i - y_i) x_i$, and $w \leftarrow w - 0.05 \times$ gradient.

sample	(x_i, y_i)	gradient	new w
1	(1, 2.1)	$(0 - 2.1)(1) = -2.1$	0.105
2	(2, 3.9)	$(0.21 - 3.9)(2) = -7.38$	0.474
3	(3, 6.2)	$(1.422 - 6.2)(3) = -14.334$	1.1907

SGD makes one cheap update per reading as data arrives.

(b) **Batch and mini-batch.** Each uses the *mean* gradient of its samples.

- **Batch:** mean gradient at $w = 0$ is $-(2.1 + 7.8 + 18.6)/3 = -9.5$, so $w_1 = 0.475$. After three batch steps $w = 1.118$, still below the SGD epoch's 1.191.
- **Mini-batch** $\{1, 2\}$: mean gradient $-(2.1 + 7.8)/2 = -4.95$, so $w = 0.2475$. Then $\{3\}$: $(0.7425 - 6.2)(3) = -16.37$, so $w = 1.0661$.

(c) **Exact answer.** Setting $\sum_i (w x_i - y_i) x_i = 0$:

$$w^* = \frac{\sum x_i y_i}{\sum x_i^2} = \frac{28.5}{14} = 2.0357$$

After 50 epochs: constant α gives $w = 2.0459$ (off by 0.010); the decaying schedule gives $w = 2.0374$ (off by 0.0016), six times closer.

Why the constant rate stalls. Each sample pulls w towards its own best fit (2.1, 1.95, 2.07). With a fixed α the last pull always wins a little, so SGD settles at a slightly biased point. A shrinking α makes those pulls fade.

(d) New reading (4, 8.1). Continue from the constant-rate model $w = 2.0459$:

$$\text{gradient} = (2.0459 \times 4 - 8.1)(4) = 0.334, \quad w \leftarrow 2.0459 - 0.05(0.334) = \boxed{2.0292}$$

One update incorporates the new reading.

Take-away

SGD trades exact gradients for cheap updates. A decaying learning rate helps it settle.

Problem A4

Application ★★

A factory makes cylindrical cans holding $V = 500 \text{ cm}^3$. Less metal means lower cost, so minimise the surface area

$$A(r) = 2\pi r^2 + 2\pi rh, \quad h = \frac{V}{\pi r^2} \implies A(r) = 2\pi r^2 + \frac{1000}{r}.$$

- (a) Find r^* , h^* and A^* exactly.
- (b) Run GD from $r_0 = 2 \text{ cm}$ with $\alpha = 0.01$. How many steps until $|A'| < 10^{-6}$?
- (c) What is the largest stable α near r^* ? Try $\alpha = 0.06$ from $r_0 = 4$.

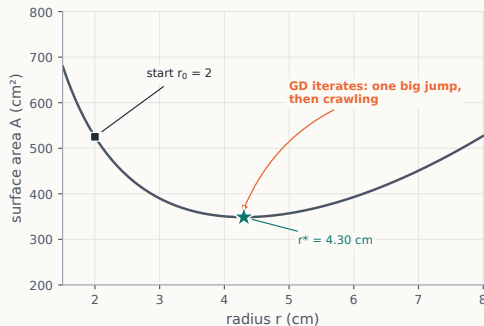
(a) Exact optimum.

$$A'(r) = 4\pi r - \frac{1000}{r^2} = 0 \implies r^* = \left(\frac{250}{\pi}\right)^{1/3}$$

$$r^* = 4.301 \text{ cm}, \quad h^* = 8.603 \text{ cm} = 2r^*$$

$A^* = 348.73 \text{ cm}^2$. Since $A''(r) = 4\pi + 2000/r^3 > 0$ for all $r > 0$, this is the only minimum.

Optimal height equals diameter.



(b) GD from $r_0 = 2$, $\alpha = 0.01$. At $r = 2$ the slope is $A'(2) = 25.13 - 250 = -224.87$, so

$$r_1 = 2 - 0.01(-224.87) = 4.2487.$$

step	1	2	3	32
r_k (cm)	4.2487	4.2688	4.2811	4.3013
$A(r_k)$ (cm ²)	348.787	348.754	348.742	348.734

The first step is large because the slope is steep where the can is thin. GD then converges, and $|A'| < 10^{-6}$ after 32 steps.

(c) Stability. Near r^* the error factor is $1 - \alpha A''(r^*)$, with $A''(r^*) = 37.70$:

$$0 < \alpha < \frac{2}{37.70} = 0.0531$$

With $\alpha = 0.06$ the factor is $1 - 0.06(37.70) = -1.26$, and $|-1.26| > 1$:

$$4 \rightarrow 4.734 \rightarrow 3.842 \rightarrow 5.010 \rightarrow 3.623 \rightarrow 5.463 \rightarrow \dots$$

The radius oscillates between narrow and wide cans, with A *rising* each time. It ends up flipping between $r \approx 2.9$ and 7.8 forever without reaching r^* .

Take-away

A rate 13% above the limit fails here. Check $\alpha < 2/f''$ before trusting a run.

Problem A5**Application** ★ ★ ★

Five students: hours studied $x = (1, 2, 3, 4, 5)$, result $y = (0, 0, 1, 0, 1)$ (1 means pass). Model $p = \sigma(wx + b) = 1/(1 + e^{-(wx+b)})$ with the mean cross-entropy loss

$$L(w, b) = -\frac{1}{m} \sum_i [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)].$$

- (a) The gradients are $\partial L / \partial w = \frac{1}{m} \sum (p_i - y_i) x_i$ and $\partial L / \partial b = \frac{1}{m} \sum (p_i - y_i)$. Take two GD steps from $(0, 0)$ with $\alpha = 0.5$.
- (b) Why can't we just solve $\nabla L = 0$ by hand?
- (c) Run GD to convergence. Predict the chance of passing after 3.5 hours, and find the pass/fail boundary.

(a) Step 1 from $(w, b) = (0, 0)$. Every $p_i = \sigma(0) = 0.5$, and $L = \ln 2 = 0.6931$.

$$p_i - y_i = (0.5, 0.5, -0.5, 0.5, -0.5)$$

$$\frac{\partial L}{\partial w} = \frac{1}{5}(0.5 + 1 - 1.5 + 2 - 2.5) = -0.1, \quad \frac{\partial L}{\partial b} = \frac{1}{5}(0.5) = 0.1$$

$$(w, b)_1 = (0 + 0.05, 0 - 0.05) = (0.05, -0.05)$$

Step 2. Now $p = (0.500, 0.512, 0.525, 0.537, 0.550)$ and $L = 0.6850$, lower than before.
The gradient is $(-0.00024, 0.1249)$, so

$$(w, b)_2 = (0.05012, -0.11247)$$

(b) No closed form. Setting the gradients to zero gives

$$\sum_i (\sigma(wx_i + b) - y_i) = 0, \quad \sum_i (\sigma(wx_i + b) - y_i)x_i = 0.$$

Here w and b appear inside exponentials, so the equations cannot be rearranged for them. GD handles this case: the loss is smooth, but “slope = 0” cannot be solved.

The loss is *convex* (its Hessian is positive definite), so there is one minimum, and GD from any start reaches it. Unlike M1, the starting point does not matter.

(c) After 20 000 steps, the gradient is essentially zero ($\approx 4 \times 10^{-16}$):

$$w = 1.0904, \quad b = -3.8940$$

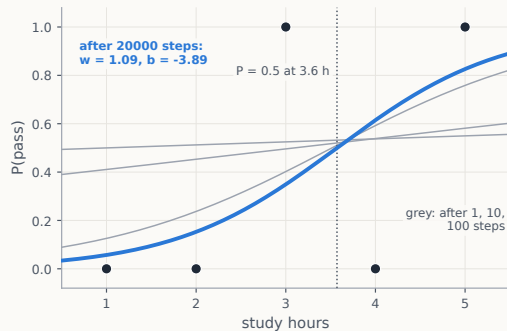
with $L = 0.4844$. After 3.5 hours:

$$p = \sigma(1.0904 \times 3.5 - 3.8940) = 0.481$$

The boundary is where $p = 0.5$, i.e. $w x + b = 0$:

$$x = -b/w = 3.57 \text{ hours}$$

Above about $3\frac{1}{2}$ hours, the model predicts a pass.





References

-
- [1] Larry Armijo. **Minimization of functions having Lipschitz continuous first partial derivatives.** *Pacific Journal of Mathematics*, 16(1):1–3, 1966.
 - [2] Jonathan Barzilai and Jonathan M. Borwein. **Two-point step size gradient methods.** *IMA Journal of Numerical Analysis*, 8(1):141–148, 1988.
 - [3] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. **Optimization methods for large-scale machine learning.** *SIAM Review*, 60(2):223–311, 2018.
 - [4] Stephen Boyd and Lieven Vandenberghe. **Convex Optimization.** Cambridge University Press, 2004.
 - [5] Sébastien Bubeck. **Convex optimization: Algorithms and complexity.** *Foundations and Trends in Machine Learning*, 8(3–4):231–357, 2015.

-
- [6] Augustin-Louis Cauchy. **Méthode générale pour la résolution des systèmes d'équations simultanées.** *Comptes Rendus Hebdomadaires des Séances de l'Académie des Sciences*, 25:536–538, 1847.
 - [7] Steven C. Chapra and Raymond P. Canale. **Numerical Methods for Engineers.** McGraw-Hill Education, 8th edition, 2020.
 - [8] Jeremy M. Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter, and Ameet Talwalkar. **Gradient descent on neural networks typically occurs at the edge of stability.** In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2103.00065.
 - [9] Gabriel Goh. **Why momentum really works.** Distill, 2017.
 - [10] Andreas Griewank and Andrea Walther. **Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation.** SIAM, 2nd edition, 2008.

-
- [11] Hamed Karimi, Julie Nutini, and Mark Schmidt. **Linear convergence of gradient and proximal-gradient methods under the Polyak–Łojasiewicz condition.** In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2016)*, volume 9851 of *Lecture Notes in Computer Science*, pages 795–811. Springer, 2016.
- [12] Diederik P. Kingma and Jimmy Ba. **Adam: A method for stochastic optimization.** In *3rd International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.
- [13] Jason D. Lee, Max Simchowitz, Michael I. Jordan, and Benjamin Recht. **Gradient descent only converges to minimizers.** In *Proceedings of the 29th Conference on Learning Theory (COLT)*, volume 49 of *Proceedings of Machine Learning Research*, pages 1246–1257, 2016.

-
- [14] Laurent Lessard, Benjamin Recht, and Andrew Packard. **Analysis and design of optimization algorithms via integral quadratic constraints.** *SIAM Journal on Optimization*, 26(1):57–95, 2016.
- [15] Yurii Nesterov. **Lectures on Convex Optimization.** Springer, 2nd edition, 2018.
- [16] Yurii E. Nesterov. **A method for solving the convex programming problem with convergence rate $O(1/k^2)$.** *Soviet Mathematics Doklady*, 27(2):372–376, 1983.
- [17] Jorge Nocedal and Stephen J. Wright. **Numerical Optimization.** Springer Series in Operations Research and Financial Engineering. Springer, 2nd edition, 2006.
- [18] Boris T. Polyak. **Some methods of speeding up the convergence of iteration methods.** *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

-
- [19] Herbert Robbins and Sutton Monro. **A stochastic approximation method.** *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [20] Weijie Su, Stephen Boyd, and Emmanuel J. Candès. **A differential equation for modeling Nesterov’s accelerated gradient method: Theory and insights.** *Journal of Machine Learning Research*, 17(153):1–43, 2016.